Waypoint-2: A Separable Causal Video World Model For Real-Time Streaming On Consumer Devices

Rajit Rajpal* **Shahbuland Matiana*** **Liew Wei Pyn*** **Anmol Agarwal***
Ryan Craig Andrew Lapp Jerry Xiong Mithun Hunsur Scottie Fox Chris Foster
Elizabeth DaCruz Carson Poole Irene Park David Rossi Spencer Frazier Louis Castricato
Overworld
*Equal contribution

Project Page

Abstract

We present **Waypoint-2**: a real-time video world model trained on gameplay and egocentric real-world video that generates interactive 720p video from text, keyboard and mouse input on consumer GPUs. Waypoint-2 is built on Separable Causal Diffusion (SCD), which splits a causal video diffusion model into a deep frame-causal encoder that runs once per frame and a lightweight decoder that attends only within the current frame and runs at every denoising step. This split moves most of the model's capacity out of the per-step cost, so a larger model fits within a fixed frame-rate budget. It also enables a training curriculum that starts with text-to-image pretraining of the decoder and moves on to video, first bidirectionally and then causally. On an RTX 5090, Waypoint-2 generates 720p video at 75 FPS including decoding. Measured with the same benchmark, it is faster than Waypoint-1.5 on every GPU we tested despite its larger model and higher resolution, and it follows keyboard and mouse input more closely.



Figure 1: **Waypoint-2 teaser.** Two example autoregressive rollouts generated in real time on a single consumer GPU (top and bottom rows). The 1.49B parameter model generates high-resolution video conditioned on text, keyboard and mouse input.

Preprint.

1 Introduction

A video world model generates the next frames of a scene from what has been seen so far and what the user does. When the user is a person at a keyboard and mouse, the model has to respond within the time of a single frame, every frame, for as long as they keep playing. Learned models can now generate explorable worlds in this way [5, 17, 19, 31], but much of this progress assumes datacenter hardware (Table 1). Waypoint-1.5 [25] showed that a 1.28B-parameter model can run in real time on consumer GPUs. This report asks how far that can be pushed: how much model fits inside a real-time budget on the hardware people already own?

The budget is tight. To show 60 frames per second when each latent frame covers four video frames, a model must produce a new latent frame every 66.7 ms, including the time to decode it into pixels. An autoregressive diffusion model spends that time on several denoising passes per frame. In a single-stream causal transformer, the design used by Waypoint-1.5, every layer runs at every denoising step and attends to the cached context, so the cost of a frame grows with the product of depth and step count. Few-step distillation [38, 16] reduces the number of steps but not the cost of each one, so the size of the model is still capped by the budget.

Waypoint-2 is built on Separable Causal Diffusion (SCD) [3]. SCD separates two jobs that a single-stream model does together at every step: working out what the next frame should contain, and producing the frame itself. A deep encoder with frame-causal attention reads the past frames, the player's input and the text prompt, and produces a context embedding for the next frame. It runs once per frame, and its keys and values are cached so that earlier frames are never encoded again. A shallow decoder then denoises the next frame from that embedding. The decoder attends only within the frame it is generating, which keeps it cheap, and it is the only part of the model that runs at every denoising step. Capacity placed in the encoder is therefore paid for once per frame rather than once per step, so a larger model fits in the same budget (Section 4). Waypoint-2 generates 720p video at 75 FPS on an RTX 5090, including decoding.

The split also lets us train the model in stages. Because the decoder only ever sees one frame, it is an image generator, and a learned beginning-of-sequence (BOS) token, which stands in for the missing previous frame, lets us pretrain it on text-to-image data before the encoder exists. The encoder is then added and trained on video, bidirectionally at first and only later causally (Section 6). Waypoint-2 is conditioned on a text prompt, which can change during a rollout, and on raw keyboard and mouse input read once per displayed frame. Many gaming world models instead express actions as camera motion or pose [19, 42, 36, 31, 28], which needs pose labels and leaves out actions that do not move the camera. Only a few [25, 34, 24, 22, 31, 27] take raw input directly. We also train on real-world egocentric video alongside gameplay to improve generalization capabilities of our model.

Finally, the factorization gives the encoder an unusual role. It never sees the frame it helps to generate, and it is the only path by which the past reaches the decoder. Its output must therefore capture whatever about the next frame can be predicted from the past and the player's input, while the generative decoder supplies what cannot. This is close in spirit to predictive self-supervised learning.

Table 1: Comparison with other real-time world models. Numbers are as reported by each work, on different hardware and settings. Video FPS counts output frames. For the Waypoint models, measured in BF16 with the same benchmark, it excludes decoding.

Model	Resolution	Parameters	Action input	Hardware	Video FPS
HY-World 1.5 [17, 27]	720p	8.3B	keys, camera pose	8 H800	24
Hunyuan-GameCraft [19]	720p	13B	camera motion	not stated	6.6
LingBot-World-Fast [31]	480p	14B	keys, camera pose	1 GPU node	16
Genie 3 [5]	720p	not disclosed	navigation, text	8 H100	24
Matrix-Game 3.0 [34]	720p	5B	keyboard, mouse	8 GPUs + 1 for decoding	40
Matrix-Game 3.5 [24]	1280×704	5B	camera pose, text	1 H100	20
Matrix-Game 4.0 [40]	480p	5B	engine render, text	1 RTX 5090	83 (FP8)
Waypoint-1.5 [25]	512×1024	1.28B	keyboard, mouse	1 RTX 5090	66.3
Waypoint-2	**720p**	**1.49B**	**keyboard, mouse, text**	**1 RTX 5090**	**89.8**

Our contributions are as follows:

1. Waypoint-2, to our knowledge the first interactive world model built on SCD, together with an analysis of how the factorization raises model capacity under a real-time budget.
2. A training curriculum that pretrains the decoder on text-to-image data, inserts the encoder so that it initially passes no information between frames, moves from bidirectional to causal video training, and ends with long-context training with a pinned BOS token.
3. A real-time 720p world model conditioned on text and on raw keyboard and mouse input, with throughput measured on consumer GPUs. It runs at 75 FPS on an RTX 5090 including decoding, faster than Waypoint-1.5 on every GPU we tested.

2 Related Work

Early world models trained a policy inside a learned model of the environment [14]. Diffusion models later turned such learned simulators into high-fidelity models of single games [2, 32], and the same recipe has since been scaled to open-ended worlds [10, 5, 17, 31, 42, 29], to physical AI [23, 30], and to multiplayer and agent-training settings [26, 1]. Many of these models take the user's input as camera motion or pose [19, 42, 36, 31, 28]. Matrix-Game [34, 24], Yume1.5 [22], LingBot-World (Act) [31], WorldPlay [27] and Waypoint-1.5 [25] instead condition on raw keyboard and mouse input, and Waypoint-1.5 showed that such a model can run in real time on consumer GPUs. Matrix-Game 4.0 [40] also runs in real time on a consumer GPU (Table 1), but its video model takes no user actions and only translates a game engine's low-poly render, a lighter video-to-video task that it runs in one denoising step instead of four, with fewer tokens per frame (405 against 512), a shorter context (21 latent frames against 68) and FP8 instead of BF16.

Video diffusion models first generated whole clips at once, with every frame attending to every other [7, 15, 33]. Interactive use needs one frame at a time. Self-Forcing [16] and other works [9, 37] distill many-step bidirectional models into few-step causal models using distribution matching distillation (DMD) [38, 39], and condition on their own rollouts during training to reduce autoregressive drift. These methods cut the number of denoising steps, but every step still runs the whole network over the frame and its context. Bai et al. [3] found much of this work to be redundant: early layers compute nearly the same features at every step, and deep layers barely attend across frames. Their Separable Causal Diffusion (SCD) therefore runs a causal encoder once per frame and a lightweight decoder, which attends only within the frame, at every step. SCD was demonstrated on low-resolution video trained from scratch and by converting a pretrained text-to-video model. Waypoint-2 is, to our knowledge, the first interactive world model built on it.

3 Problem Formulation

A video autoencoder maps a video to a sequence of latent frames $x_1, \dots, x_T$, each covering $\tau = 4$ consecutive video frames (Appendix A). The player's input is read once per video frame, so latent frame t comes with τ input readings $a_t = (a_t^{(1)}, \dots, a_t^{(\tau)})$, each recording which keys are held, how far the mouse moved and the direction of any scroll. A text prompt c_t describes the scene and may change during a rollout, and r is the frame rate of the source video. A world model generates latent frames one at a time, each conditioned on everything that came before it:

$$p_\theta(x_{1:T}) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t},\, a_{\le t},\, c_{\le t},\, r)\,. \tag{1}$$

A rollout can start from one or more seed frames or from the prompt alone.

We model each conditional distribution in Equation (1) with flow matching. A clean latent frame x_t is mixed with Gaussian noise at a level $\sigma \in [0, 1]$,

$$x_t^\sigma = (1-\sigma)\, x_t + \sigma\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I), \tag{2}$$

and a network v_θ learns to predict the velocity $\varepsilon - x_t$ from the noisy frame, the noise level and the conditioning:

$$\mathcal{L} = \mathbb{E}_{t,\sigma,\varepsilon}\left[\left\| v_\theta(x_t^\sigma, \sigma \mid x_{<t}, a_{\le t}, c_{\le t}, r) - (\varepsilon - x_t)\right\|^2\right]. \tag{3}$$

During training, σ is drawn uniformly and then shifted toward higher noise levels, $\sigma \leftarrow 3\sigma/(1+2\sigma)$, as is common at high resolution [11].

The Real-Time Budget. A world model generates f_ℓ latent frames per second and shows $f_p = \tau f_\ell$ video frames per second. Waypoint-2 reads one input per video frame, so its control rate is also f_p, whereas Waypoint-1.5 read input once per latent frame [25]. Showing 60 frames per second requires $f_\ell \geq 15$, which leaves 66.7 ms per latent frame for all N denoising passes and decoding. Our goal is to fit as much model capacity as possible within this budget (Section 4).

4 Separable Causal Diffusion

SCD [3] splits the velocity network of Equation (3) into an encoder E and a decoder D with different jobs (Figure 2):

$$h_t = E(x_{<t},\, a_{\leq t},\, c_{\leq t},\, r)\,, \qquad \hat{v}_t = D(x_t^\sigma,\, \sigma,\, h_t)\,. \tag{4}$$

The encoder reads the history and produces a context embedding h_t for the next frame. It carries all information across time and runs once per latent frame. The decoder denoises frame t given h_t. It never looks beyond the current frame, and it is the only part of the model that runs at every denoising step. Our use of SCD departs from the original in how the encoder is trained. Bai et al. [3] train it causally, whereas we first pretrain it with bidirectional attention across frames and make it causal only afterwards (Section 6). Table 3 lists the configuration of both.

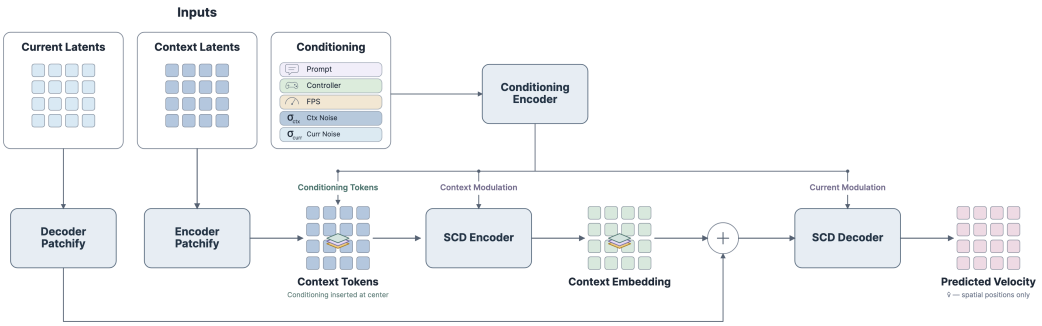


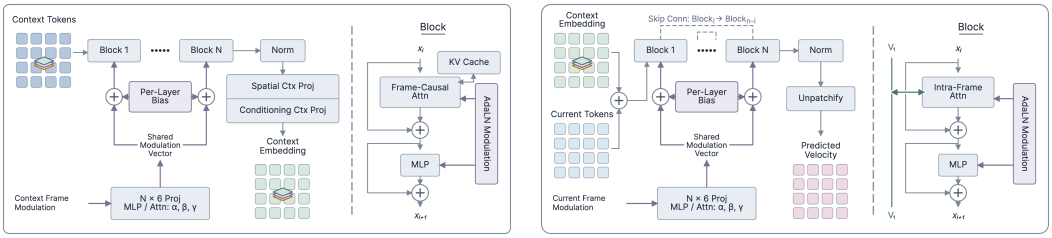
Figure 2: **Waypoint-2 architecture.** The conditioning encoder turns the prompt, controls, frame rate and noise levels into conditioning tokens and modulation vectors. The SCD encoder processes past latents and per-frame conditioning tokens with frame-causal attention and a KV cache, once per frame. Its context embedding is added to the embedded noisy latent of the current frame, and the SCD decoder, which attends only within that frame, predicts the velocity at every denoising step.

Context Encoder

The encoder turns the history into a context embedding for the next frame (Figure 3a). Its input is shifted by one frame: the slot that produces the context for frame t holds frame $t-1$, and the first slot holds only the BOS token (described below). Each slot also carries the conditioning tokens of the frame it predicts. Attention is always bidirectional within a frame. Across frames, it is bidirectional during pretraining, when the encoder sees a noisy version of every frame in the clip. It becomes causal afterwards, and the encoder then sees only clean past frames, within a window of recent frames plus a few frames pinned from the start of the rollout (Section 6). In the causal model, the output for frame t depends on $x_{<t}$ and $a_{\leq t}$ but never on x_t, and keys and values are cached so that each frame is encoded exactly once. The final states are normalized and passed to the decoder through two linear projections, one for the spatial tokens and one for the conditioning tokens.

Intra-Frame Decoder

The decoder turns the context embedding into a frame (Figure 3b). At each denoising step it adds an embedding of the noisy latent to the context embedding and processes the result together with the frame's conditioning tokens. Attention never leaves the frame, so the cost of a decoder step does not depend on the length of the rollout. Long skip connections join early and late decoder layers, as in a U-Net, and values from the first layer are mixed into later ones (value residual learning [41]). The decoder is modulated by the noise level and outputs the velocity of Equation (3).



(a) SCD encoder. (b) SCD decoder.

Figure 3: **SCD encoder and decoder.** (a) The encoder processes the context tokens with frame-causal attention, caching keys and values, and projects its normalized output into the context embedding. (b) The decoder adds the context embedding to the embedded noisy tokens of the current frame and predicts the velocity, with intra-frame attention, skip connections, and a value residual.

Conditioning

Waypoint-2 is conditioned on the signals below (Figure 4). Text and controls enter as tokens attached to each frame. The remaining signals, together with summaries of the text and controls, modulate every layer through AdaLN [43].

- **Text.** Qwen 3.5 encodes the prompt, and a small transformer with learned register tokens condenses its hidden states into a fixed set of text tokens and a global vector. The prompt can change during a rollout.
- **Controls.** One control token per video frame, built from the held keys, the mouse displacement and the scroll direction. Clips without recorded input use a learned null control.
- **Frame rate.** An embedding of the source frame rate, so that clips recorded at different rates can be mixed.
- **Noise level.** The noise level of the frame being denoised modulates the decoder, and that of the context frames modulates the encoder.

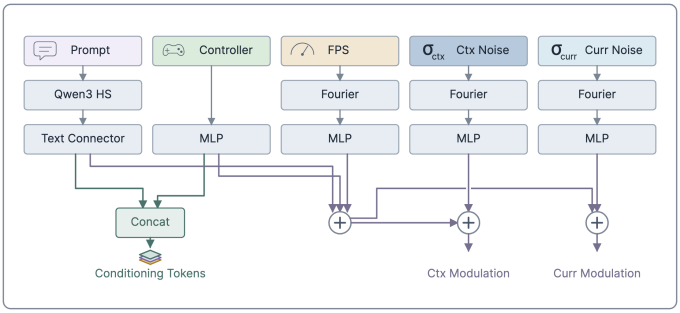


Figure 4: **Conditioning encoder.** The text and control embeddings form the conditioning tokens of each frame. Their summaries are added to embeddings of the frame rate and the noise level (of the context for the encoder, of the current step for the decoder) to modulate each network.

The BOS Token: Images as One-Frame Videos

The first frame of a rollout has no previous frame for the encoder to read. We fill that slot with a learned BOS embedding, which is also added to every other encoder input. This has two consequences. Generating the first frame means generating an image from the prompt alone, so T2I generation is the one-frame case of the world model and uses the same parameters. This is what makes decoder-only pretraining possible (Section 6). A rollout can also start from text without a seed image.

Cost Analysis: Where to Spend Capacity?

At inference, each latent frame takes one encoder pass over the newest frame, which appends that frame's keys and values to the cache, then N decoder passes and one autoencoder decode. Ignoring the autoencoder, the cost of a latent frame is $C_{\text{SCD}} = C_E + N\,C_D$, where C_E is the cost of the encoder pass and C_D that of one decoder pass. A single-stream model like Waypoint-1.5 instead runs every layer at every step, each layer attending to the full context, so its cost is multiplied by N.

Table 2 puts numbers on this for the configuration in Table 3, with a full cache. An encoder pass costs 4.08 TFLOPs, of which 80% is attention over the cached context, and a decoder step costs 0.49 TFLOPs. With the 4 denoising steps used at inference, a latent frame costs 6.0 TFLOPs. Running the same 46 layers as a single stream would cost 25.2 TFLOPs, 4.2 times as much. Put differently, at Waypoint-2's cost per frame, a single-stream model with the same layer dimensions could have only about 11 layers instead of 46.

Table 2: Forward cost per latent frame in TFLOPs, at steady state with a full context of 64 frames plus 4 pinned frames. We count the matrix multiplications in the transformer layers. The single-stream model has the same 46 layers but runs all of them at every step, each attending to the full context.

	$N = 1$	$N = 2$	$N = 4$
SCD (encoder once, decoder N times)	4.57	5.06	6.04
Single stream, same layers	6.30	12.60	25.20
Ratio	1.4×	2.5×	4.2×
Single-stream depth at equal cost	34	19	11

Two design consequences follow. Depth belongs in the encoder, and the decoder should be as shallow as quality allows: in Waypoint-2 the encoder holds 52% of the parameters, while only the decoder's 30% run at every step. The advantage also grows with the number of steps and with the context length, because only the encoder attends to the context. Conversely, with fewer steps the advantage shrinks, since the encoder term remains. The same asymmetry helps during training, where one encoder pass can serve several decoder noise draws (Section 6). The number of tokens per frame, set by the autoencoder (Appendix A), scales both terms. Measured on real GPUs (Figure 5), the split follows the FLOP count of Table 2: the encoder pass takes 63 to 66% of the time of a latent frame, close to its 68% share of the FLOPs, the four decoder steps take about a quarter, and autoencoder decoding the remaining tenth.

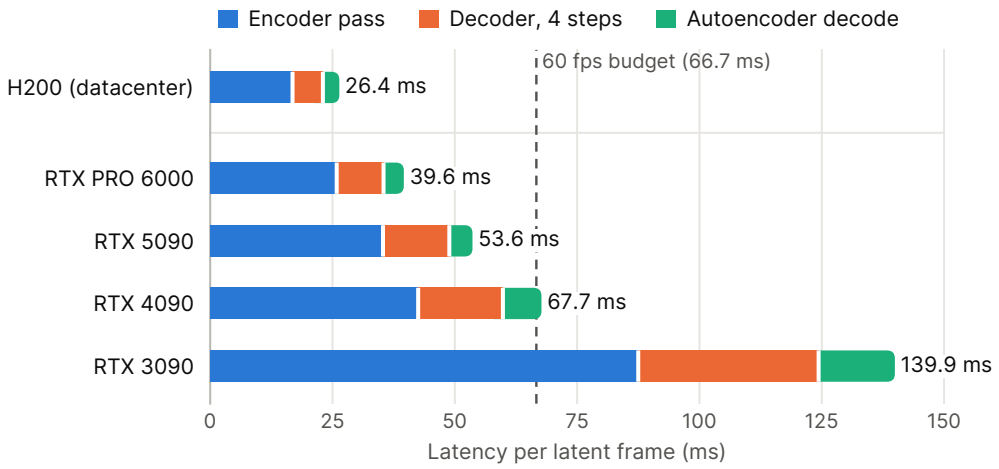


Figure 5: **Where the time goes.** Latency per latent frame on consumer GPUs and, for reference, a datacenter H200, split into the encoder pass, the four decoder steps and autoencoder decoding. The dashed line marks the 66.7 ms budget for 60 frames per second.

Table 3: Waypoint-2 model configuration.

Component	Value
Trainable parameters	1.49B, excluding the frozen text encoder and the autoencoder
Encoder	30 layers, width 1536, 24 query and 12 key-value heads, 0.78B parameters
Decoder	16 layers, width 1536, 12 heads, long skip connections, 0.45B parameters
MLP	expansion $4\times$, SiLU
Cross-frame attention	frame-causal, window of 64 latent frames plus 4 pinned frames
Text encoder	Qwen3.5-4B (frozen), hidden states from 9 layers
Text connector	4 layers, 64 thinking registers, 28 output registers
Tokens per latent frame	512 (480 spatial, 28 text, 4 control)
Output resolution	720×1280
Latent frame	$15 \times 32 \times 128$, covering 4 perceptual frames
Control rate	one control vector per perceptual frame
Positional encoding	2D GGRoPE (space) and 1D RoPE (time)
Noise conditioning	AdaLN with a shared projection and per-layer biases

5 Data

Waypoint-2 is trained on Owl-Control, an internal dataset of gameplay recorded together with keyboard and mouse input, whose collection and preprocessing are described in Waypoint-1.5 [25]. We also mix in real-world egocentric footage.

6 Training

Waypoint-2 is trained in four stages that move from single images to long causal rollouts (Table 4 and Figure 6). Each stage starts from the weights of the previous one. All stages use the flow matching objective of Equation (3), with NorMuon [20, 18] for matrix-shaped weights and AdamW for the rest. Appendix B lists the hyperparameters of each stage.

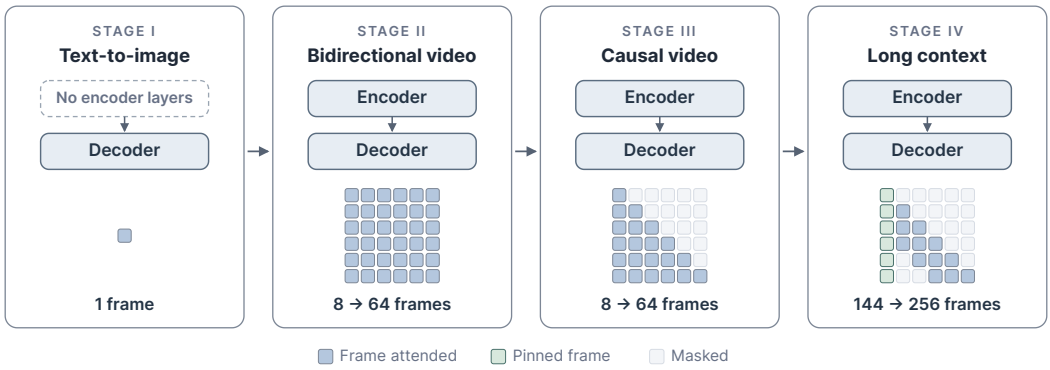


Figure 6: **Training curriculum.** The decoder is first trained alone on images, and the encoder is added in Stage II. Grids show which frames each frame can attend to, with rows for the frame being generated and columns for the frames it sees. Attention across frames moves from bidirectional to causal, and the last stage trains with a rolling window and pinned frames, as at inference. Clip lengths are in latent frames.

Stage I: Text-to-Image Pretraining of the Decoder

The first stage trains the decoder and the text pathway on images. The encoder has no layers yet, so the decoder sees only the BOS context and the conditioning tokens, and the model is a text-to-image diffusion transformer (Section 4). Images are trained on the same 720p latent grid that the video model uses later. We add the Self-Flow representation loss [8], which aligns intermediate decoder features with deeper features of an exponential moving average (EMA) of the model that sees a less

noisy version of the input. The prompt is dropped for a fraction of samples to enable classifier-free guidance.

Stage II: Bidirectional Video Pretraining

We then insert the encoder. Its AdaLN gates start at zero, and so does the projection that feeds frames into it, since that projection received no gradient in Stage I, where every token sits at the first frame position. At the start of this stage no information passes between frames, and each frame is generated from BOS and its own conditioning, as an image was in Stage I. Temporal coupling is learned from there.

Attention across frames is bidirectional in this stage. All frames of a clip share one noise level and the encoder sees their noisy versions, so the model is a bidirectional video diffusion model split into an encoder and a decoder. Up to two leading frames are kept clean, which teaches the model to continue from given frames. Training moves from short clips to long, captioned clips. Controls are dropped independently of the text, and more often in the long-clip phase, so that guidance can be applied to each condition separately. Self-Flow continues throughout. We start bidirectionally because every frame then learns from its full context, and the model can learn what worlds look like and how they move before facing the causal constraint.

Stage III: Causal Conversion

The third stage turns the bidirectional model into a causal one. The architecture stays the same, and training continues from the Stage II weights. What changes is what the encoder can see. Its attention mask becomes frame-causal: tokens still attend to every token of their own frame, but only to earlier frames, never to later ones. Because the encoder input is shifted by one frame, the context for frame t then depends only on $x_{<t}$. The encoder also stops receiving noisy frames and instead receives clean previous frames, as it does at inference, and each target frame is given its own noise level. Continued training then teaches the model to predict without access to future frames. Because the decoder works on a single frame and is cheap, each encoder pass is reused for several independent decoder noise draws, which gives the encoder more training signal per pass. As in Stage II, training moves from short to long clips. Self-Flow is no longer used.

Stage IV: Long Context with Pinned Frames

The last pretraining stage trains with the cache layout used at inference. Older frames leave a rolling window, while the BOS slot and the first few frames stay pinned and remain visible to every later frame. Temporal positions of the pinned frames are shifted so that their distance from the current frame stays within the range seen in training. Sequences grow to 256 latent frames, about 17 seconds of video.

Table 4: Training curriculum. Clip lengths are in latent frames.

Stage	Trained	Attention across frames	Clip length	Data
I. Text-to-image	decoder, text connector	none (one frame)	1	image-text pairs
II. Bidirectional video	adds the 30-layer encoder	bidirectional	8, then 64	video (Section 5)
III. Causal video	all	frame-causal	8, then 64	as in II
IV. Long context	all	frame-causal with pinned frames	144 to 256	as in II

Distillation

Finally, we distill the causal model into a student that generates each frame in four denoising steps, using a custom scheme built on recent few-step methods. The student is first trained with MeanFlow [12] to take large steps along the flow, with classifier-free guidance for text and controls folded into its predictions, so inference needs a single forward pass per step. It is then refined on its own autoregressive rollouts, as in Self-Forcing [16], using distribution matching distillation [38] against a frozen causal teacher and a trainable critic, in the manner of AnyFlow [13]. As in Solaris [26], the history of each rollout is generated without gradients and then re-encoded with gradients, so that the loss reaches the encoder through its view of the past. We leave out the remaining details of the recipe.

7 Experiments

Real-Time Performance

Table 5 reports throughput across GPUs, with Waypoint-1.5 as a reference, and Figure 5 splits the time of one latent frame between the encoder, the decoder and the autoencoder. Waypoint-2 currently runs in BF16 only.

Table 5: Throughput across consumer GPUs, in video frames per second (four per latent frame). Both models are measured with the Waypoint-1.5 benchmark, in BF16, from an empty cache and without decoding. Waypoint-1.5 numbers are from [25]. The H200 row is a datacenter GPU, shown for reference.

Hardware	Waypoint-1.5	Waypoint-2 (720p)
RTX PRO 6000 Blackwell	87.76	123.42
RTX 5090	66.28	89.80
RTX 4090	66.60	71.67
RTX 3090	21.60	35.12
RTX 3070	14.45	20.50
RTX 3060	8.80	14.28
H200 (datacenter, for reference)	168.86	234.80

Generation Quality

Table 6 and Figure 7 show text-to-image quality after Stage I. Table 7 reports video quality by training stage and conditioning mode, and Figure 8 shows qualitative rollouts.

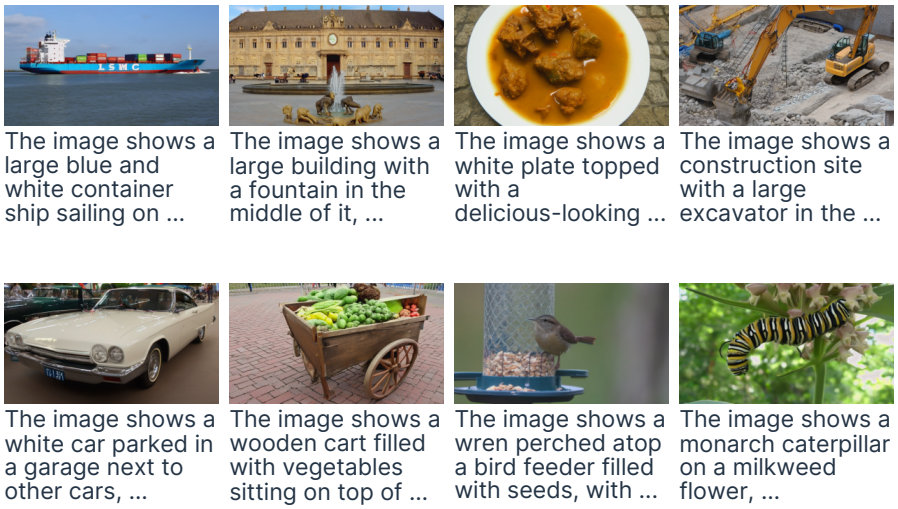


Figure 7: **Stage I samples.** Text-to-image samples from the decoder at the end of Stage I, before the encoder has any layers, at 720p with 50 steps and a guidance scale of 5. Prompts are captions of held-out images, shortened here.

Table 6: Text-to-image quality after Stage I, on 10,000 held-out prompts. Samples use 50 steps with a guidance scale of 5 and no shift of the noise schedule.

Model	FID ↓	SigLIP 2 score ↑
Real images through the autoencoder	1.49	7.51
Stage I decoder	10.29	8.55

Table 7: Video quality on 256 held-out two-second gameplay clips, as KDD / JEDi (lower is better).

Model	Text and controls	Text only	Controls only	Seedless
End of Stage II (bidirectional)	0.39 / 0.09	1.12 / 0.55	0.48 / 0.10	1.07 / 0.19
End of Stage III (causal)	1.43 / 0.18	1.18 / 0.44	1.46 / 0.18	1.47 / 0.21
End of Stage IV (long context)	1.67 / 0.22	1.76 / 0.43	1.62 / 0.21	1.66 / 0.26
Distilled model	0.45 / 0.09	0.97 / 0.34	0.53 / 0.09	1.50 / 0.24
Waypoint-1.5 [25], inference engine	n/a	n/a	0.66 / 0.22	n/a
Waypoint-2, inference engine	0.67 / 0.10	–	0.65 / 0.09	–

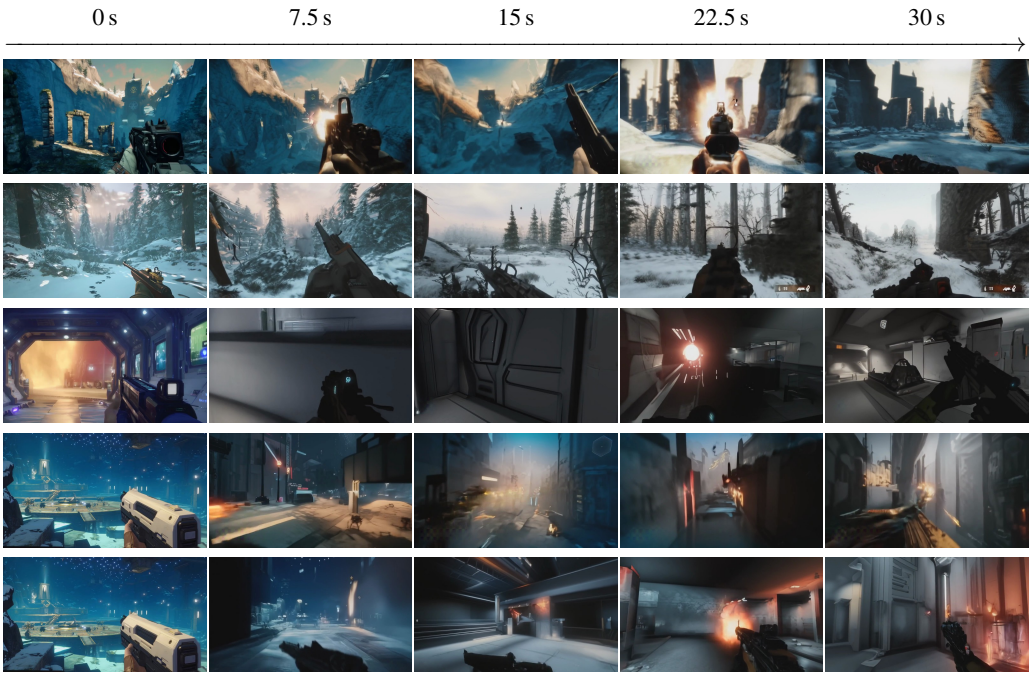


Figure 8: **Waypoint-2 autoregressive rollouts.** Each row is a 30-second rollout.

Control Fidelity

We measure how closely generated video follows the input with two models that only see the video. An inverse dynamics model (IDM) [4], trained on Owl-Control, predicts for every frame which keys are held and how the mouse moves, and Depth Anything 3 [21] estimates the camera rotation. We take 104 held-out four-second windows in which the player moves and turns, seed each model with the real first frame, and drive it with the recorded input, together with the caption for Waypoint-2, which is the only model with text conditioning. Waypoint-2 follows the input about as closely as the game itself. The IDM recovers its keys and mouse direction at least as well as on real footage, and its camera turns with the mouse (rotation agreement 0.87, against 0.88 for real video). Waypoint-1.5 follows the mouse less closely, with more than twice the direction error and lower rotation agreement.

Table 8: Control fidelity on 104 held-out four-second windows. Key F1 compares IDM's WASD predictions with input keys. Mouse direction error is median angle between predicted and input mouse motion. Rotation agreement is median correlation between yaw from DA3 and mouse motion.

Model	Key F1 ↑	Mouse direction error ↓	Rotation agreement ↑
Real video (reference)	0.86	2.0°	0.88
Waypoint-1.5 [25]	0.88	4.3°	0.64
Waypoint-2	**0.91**	**1.8°**	**0.87**

The BOS Token

We study how the encoder uses BOS in text-only rollouts of the distilled model (Figure 10), with captions of held-out clips and scripted input (walking forward while turning). Every later frame keeps attending to BOS (Figure 9a). Once the window is full, BOS receives 1.4 times the attention of an average cached frame, and the other three pinned frames about half as much. This attention comes mostly from the first four encoder layers, which give BOS 3.5 times the average share (Figure 9b), and from a few heads that resemble the attention sinks of language models [35], one of which places about half of its attention on BOS. Hiding BOS or the other pinned frames, once a plain rolling cache would have evicted them, changes the first ten seconds about as much as hiding four of the oldest frames. Over longer rollouts, however, it brings forward the onset of instability, by about 3 seconds for BOS alone and 6 seconds for all pinned frames, while hiding four old frames does not (averages over four prompts).

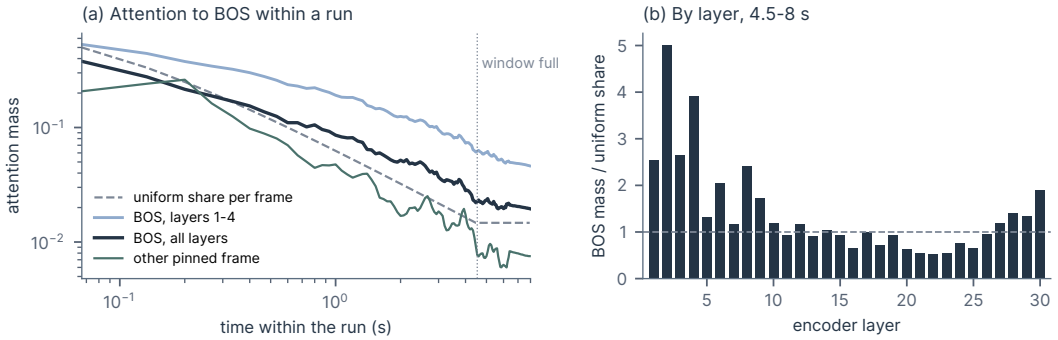


Figure 9: **BOS token.** (a) Attention that encoder places on BOS and on other pinned frames within a run, averaged over eight rollouts, with share each frame would receive under uniform attention. (b) BOS attention by encoder layer once window is full, relative to uniform share.

The SCD Encoder as a Self-Supervised Predictor

The encoder learns only through the generative loss, and its output for frame t is computed before frame t is seen. If the encoder acts as a predictor, this output should encode how the camera moves between frames $t-1$ and t. We test this with linear probes. Encoder features at the position of frame t, averaged over spatial tokens, are regressed with ridge regression onto the camera rotation and translation between the two frames, which we estimate from the real video [21]. We probe the causal model at the end of Stage III on 240 gameplay clips, split by clip into training, validation and test sets. As a baseline, the same probe is fit to the keyboard and mouse input of frame t and the few frames before it, together with the true camera motion between frames $t-2$ and $t-1$. Features from the middle of the encoder predict yaw better than this baseline (R^2 of 0.82 against 0.75), and they match it for pitch and translation (Table 9). Decodability rises with depth up to the middle of the encoder and falls toward its output, whose features serve the decoder.

Table 9: Held-out R^2 of linear probes for camera motion between consecutive latent frames. Input is keyboard and mouse input of the current and a few earlier frames, together with the true camera motion one step earlier. For yaw, the best encoder layer, chosen on validation clips, is significantly better than the input baseline (bootstrap over test clips).

Features	Yaw	Pitch	Translation
Input	0.75	**0.54**	**0.44**
Encoder, layer 5	0.58	0.30	0.17
Encoder, layer 10	0.71	0.40	0.30
Encoder, layer 15	**0.82**	0.48	0.39
Encoder, layer 20	0.80	0.50	0.43
Encoder, layer 25	0.80	0.45	0.40
Encoder output	0.70	0.40	0.36

8 Conclusion

We presented Waypoint-2, a real-time video world model built on SCD. Encoding the context once per frame and denoising with a shallow decoder lets most of the parameters sit where they run only once per frame. The same split allows a curriculum that pretrains the decoder on text-to-image data, and it turns the encoder into a predictive representation learner whose middle layers predict how the camera turns better than the controls and the previous motion do. Waypoint-2 generates 720p video from text, keyboard and mouse input at 75 FPS on an RTX 5090, follows keyboard and mouse input about as closely as real gameplay does, and after distillation into a four-step model comes close to the video quality of the bidirectional Stage II model. Memory beyond the attention window (for example retrieval from the cache), longer horizons, quantization for smaller GPUs and support for more input devices remain open directions.

References

[1] Anmol Agarwal, Pranay Meshram, Sumer Singh, Saurav Suman, Andrew Lapp, Shahbuland Matiana, Louis Castricato, and Spencer Frazier. Combat: Conditional world models for behavioral agent training. *arXiv preprint arXiv:2603.00825*, 2026.

[2] Eloi Alonso, Adam Jelley, Vincent Micheli, Anssi Kanervisto, Amos Storkey, Tim Pearce, and François Fleuret. Diffusion for world modeling: Visual details matter in atari. *Advances in Neural Information Processing Systems*, 37:58757–58791, 2024.

[3] Xingjian Bai, Guande He, Zhengqi Li, Eli Shechtman, Xun Huang, and Zongze Wu. Causality in video diffusers is separable from denoising. In *Conference on Computer Vision and Pattern Recognition 2026*, 2026. URL https://openreview.net/forum?id=p8lC6vvfR8.

[4] Bowen Baker, Ilge Akkaya, Peter Zhokhov, Joost Huizinga, Jie Tang, Adrien Ecoffet, Brandon Houghton, Raul Sampedro, and Jeff Clune. Video pretraining (VPT): Learning to act by watching unlabeled online videos. In *Advances in Neural Information Processing Systems*, volume 35. Curran Associates, Inc., 2022.

[5] Philip J. Ball, Jakob Bauer, Frank Belletti, Bethanie Brownfield, Ariel Ephrat, Shlomi Fruchter, Agrim Gupta, Kristian Holsheimer, Aleksander Holynski, Jiri Hron, Christos Kaplanis, Marjorie Limont, Matt McGill, Yanko Oliveira, Jack Parker-Holder, Frank Perbet, Guy Scully, Jeremy Shar, Stephen Spencer, Omer Tov, Ruben Villegas, Emma Wang, Jessica Yung, Cip Baetu, Jordi Berbel, David Bridson, Jake Bruce, Gavin Buttimore, Sarah Chakera, Bilva Chandra, Paul Collins, Alex Cullum, Bogdan Damoc, Vibha Dasagi, Maxime Gazeau, Charles Gbadamosi, Woohyun Han, Ed Hirst, Ashyana Kachra, Lucie Kerley, Kristian Kjems, Eva Knoepfel, Vika Koriakin, Jessica Lo, Cong Lu, Zeb Mehring, Alex Moufarek, Henna Nandwani, Valeria Oliveira, Fabio Pardo, Jane Park, Andrew Pierson, Ben Poole, Helen Ran, Tim Salimans, Manuel Sanchez, Igor Saprykin, Amy Shen, Sailesh Sidhwani, Duncan Smith, Joe Stanton, Hamish Tomlinson, Dimple Vijaykumar, Luyu Wang, Piers Wingfield, Nat Wong, Keyang Xu, Christopher Yew, Nick Young, Vadim Zubov, Douglas Eck, Dumitru Erhan, Koray Kavukcuoglu, Demis Hassabis, Zoubin Gharamani, Raia Hadsell, Aäron van den Oord, Inbar Mosseri, Adrian Bolton, Satinder Singh, and Tim Rocktäschel. Genie 3: A new frontier for world models. 2025.

[6] Ollin Boer Bohan. TAEHV: Tiny AutoEncoder for Hunyuan Video. https://github.com/madebyollin/taehv, 2025. Accessed: 2026-06-10.

[7] Tim Brooks, Bill Peebles, Connor Holmes, Will DePue, Yufei Guo, Li Jing, David Schnurr, Joe Taylor, Troy Luhman, Eric Luhman, Clarence Ng, Ricky Wang, and Aditya Ramesh. Video generation models as world simulators. 2024. URL https://openai.com/research/video-generation-models-as-world-simulators.

[8] Hila Chefer, Patrick Esser, Dominik Lorenz, Dustin Podell, Vikash Raja, Vinh Tong, Antonio Torralba, and Robin Rombach. Self-supervised flow matching for scalable multi-modal synthesis. *arXiv preprint arXiv:2603.06507*, 2026.

[9] Justin Cui, Jie Wu, Ming Li, Tao Yang, Xiaojie Li, Rui Wang, Andrew Bai, Yuanhao Ban, and Cho-Jui Hsieh. Self-forcing++: Towards minute-scale high-quality video generation. In

The Fourteenth International Conference on Learning Representations, 2026. URL https://openreview.net/forum?id=DzvPiqh23f.

[10] Etched Decart, Quinn McIntyre, Spruce Campbell, Xinlei Chen, and Robert Wachen. Oasis: A universe in a transformer. *URL: https://oasis-model. github. io*, 2(3):6, 2024.

[11] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first international conference on machine learning*, 2024.

[12] Zhengyang Geng, Mingyang Deng, Xingjian Bai, Zico Kolter, and Kaiming He. Mean flows for one-step generative modeling. *Advances in Neural Information Processing Systems*, 38: 75460–75482, 2026.

[13] Yuchao Gu, Guian Fang, Yuxin Jiang, Weijia Mao, Song Han, Han Cai, and Mike Zheng Shou. Anyflow: Any-step video diffusion model with on-policy flow map distillation. In *European Conference on Computer Vision*, pages 163–181. Springer, 2026.

[14] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2(3):440, 2018.

[15] Yoav HaCohen, Nisan Chiprut, Benny Brazowski, Daniel Shalem, Dudu Moshe, Eitan Richardson, Eran Levin, Guy Shiran, Nir Zabari, Ori Gordon, et al. Ltx-video: Realtime video latent diffusion. *arXiv preprint arXiv:2501.00103*, 2024.

[16] Xun Huang, Zhengqi Li, Guande He, Mingyuan Zhou, and Eli Shechtman. Self forcing: Bridging the train-test gap in autoregressive video diffusion. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL https://openreview.net/forum?id=mSiN7i0BYH.

[17] Team HunyuanWorld. Hy-world 1.5: A systematic framework for interactive world modeling with real-time latency and geometric consistency. *arXiv preprint*, 2025.

[18] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL https://kellerjordan.github.io/posts/muon/.

[19] Jiaqi Li, Junshu Tang, Zhiyong Xu, Longhuang Wu, Yuan Zhou, Shuai Shao, Tianbao Yu, Zhiguo Cao, and Qinglin Lu. Hunyuan-gamecraft: High-dynamic interactive game video generation with hybrid history condition. *arXiv preprint arXiv:2506.17201*, 2025.

[20] Zichong Li, Liming Liu, Chen Liang, Weizhu Chen, and Tuo Zhao. Normuon: Making muon more efficient and scalable, 2025. URL https://arxiv.org/abs/2510.05491.

[21] Haotong Lin, Sili Chen, Junhao Liew, Donny Y Chen, Zhenyu Li, Guang Shi, Jiashi Feng, and Bingyi Kang. Depth anything 3: Recovering the visual space from any views. *arXiv preprint arXiv:2511.10647*, 2025.

[22] Xiaofeng Mao, Zhen Li, Chuanhao Li, Xiaojie Xu, Kaining Ying, and Kaipeng Zhang. Yume1. 5: A text-controlled interactive world generation model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 7752–7761, 2026.

[23] NVIDIA, :, Aditi, Niket Agarwal, Arslan Ali, Jon Allen, Martin Antolini, Adeline Aubame, Alisson Azzolini, Junjie Bai, Maciej Bala, Yogesh Balaji, Josh Bapst, Aarti Basant, Mukesh Beladiya, Mohammad Qazim Bhat, Zaid Pervaiz Bhat, Dan Blick, Vanni Brighella, Han Cai, Tiffany Cai, Eric Cameracci, Jiaxin Cao, Yulong Cao, Mark Carlson, Carlos Casanova, Ting-Yun Chang, Yan Chang, Yu-Wei Chao, Prithvijit Chattopadhyay, Roshan Chaudhari, Chieh-Yun Chen, Junyu Chen, Ke Chen, Qizhi Chen, Wenkai Chen, Xiaotong Chen, Yu Chen, An-Chieh Cheng, Click Cheng, Xiu Chia, Jeana Choi, Chaeyeon Chung, Wenyan Cong, Yin Cui, Magdalena Dadela, Nalin Dadhich, Wenliang Dai, Joyjit Daw, Alperen Degirmenci, Rodrigo Vieira Del Monte, Robert Denomme, Sameer Dharur, Marco Di Lucca, Ke Ding, Wenhao Ding, Yifan Ding, Yuzhu Dong, Nicole Drumheller, Yilun Du, Aigul Dzhumamuratova, Aleksandr

Efitorov, Hamid Eghbalzadeh, Naomi Eigbe, Imad El Hanafi, Hassan Eslami, Benedikt Falk, Jiaojiao Fan, Jim Fan, Amol Fasale, Sergiy Fefilatyev, Liang Feng, Francesco Ferroni, Sanja Fidler, Xiao Fu, Vikram Fugro, Prashant Gaikwad, TJ Galda, Katelyn Gao, Yihuai Gao, Wenhang Ge, Sreyan Ghosh, Arushi Goel, Vivek Goel, Akash Gokul, Rama Govindaraju, Jinwei Gu, Miguel Guerrero, Elfie Guo, Aryaman Gupta, Siddharth Gururani, Hugo Hadfield, Song Han, Ankur Handa, Zekun Hao, Mohammad Harrim, Ali Hassani, Nathan Hayes-Roth, Yufan He, Chris Helvig, Cyrus Hogg, Madison Huang, Michael Huang, Sophia Huang, Yufan Huang, Jacob Huffman, DeLesley Hutchins, Suneel Indupuru, Boris Ivanovic, Arihant Jain, Joel Jang, Ryan Ji, Yanan Jian, Dongfu Jiang, Jingyi Jin, Atharva Joshi, Nikhilesh Joshi, Pranjali Joshi, Andy Ju, Jaehun Jung, Weiwei Kang, Scott Kassekert, Jan Kautz, Ashna Khetan, Julia Kiczka, Slawek Kierat, Gwanghyun Kim, Kuno Kim, Sunny Kim, Kezhi Kong, Xin Kong, Zhifeng Kong, Tomasz Kornuta, Egor Krivov, Hui Kuang, Saurav Kumar, Chia-Wen Kuo, George Kurian, Wojciech Kutak, JF Lafleche, Himangshu Lahkar, Omar Laymoun, Jayjun Lee, Sanggil Lee, Gabriele Leone, Boyi Li, Freya Li, Jiajun Li, Jinfeng Li, Ling Li, Pengcheng Li, Shangru Li, Tingle Li, Xiaolong Li, Xuan Li, Zhaoshuo Li, Zhiqi Li, Hao Liang, Maosheng Liao, Chen-Hsuan Lin, Tsung-Yi Lin, Ming-Yu Liu, Sifei Liu, Zihan Liu, Hai Loc Lu, Xiangyu Lu, Alice Luo, Ruipu Luo, Wenjie Luo, Jiangran Lyu, Martin Ding Ma, Nic Ma, Qianli Ma, Dawid Majchrowski, Louis Marcoux, Miguel Martin, Qing Miao, Ashkan Mirzaei, Shreyas Misra, Kaichun Mo, Durra Mohsin, Hyejin Moon, Pawel Morkisz, Saeid Motiian, Kirill Motkov, Seungjun Nah, Yashraj Narang, Deepak Narayanan, Thabang Ngazimbi, Julian Ouyang, Shubham Pachori, David Page, Yatian Pang, Sehwi Park, Mahesh Patekar, Mostofa Patwary, Marco Pavone, Trung Pham, Wei Ping, Soha Pouya, Shrimai Prabhumoye, Varun Praveen, Delin Qu, Hesam Rabeti, Morteza Ramezanali, Marilyn Reeb, Xuanchi Ren, Kristen Rumley, Wojciech Rymer, Jun Saito, Yeongho Seol, John Shao, Piyush Shekdar, Tianwei Shen, Humphrey Shi, Min Shi, Stella Shi, Kevin Shih, Mohammad Shoeybi, Mateusz Sieniawski, Shuran Song, Alexander Sotelo, Amir Sotoodeh, Sunil Srinivasa, Vignesh Srinivasakumar, Bartosz Stefaniak, Rahul Heinrich Steiger, Shangkun Sun, Jiaxiang Tang, Shitao Tang, Yangyang Tang, Yue Tang, Tolou Tavakkoli, Kayley Ting, Krzysztof Tomala, Wei-Cheng Tseng, Jibin Varghese, Sergei Vasilev, Thomas Volk, Raju Wagwani, Roger Waleffe, Andrew Z. Wang, Boxiang Wang, Haoxiang Wang, Qiao Wang, Shihao Wang, Shijie Wang, Ting-Chun Wang, Yan Wang, Yu Wang, Rohit Watve, David Wehr, Fangyin Wei, Xinshuo Weng, Jay Zhangjie Wu, Kedi Wu, Hongchi Xia, Summer Xiao, Tianjun Xiao, Kevin Xie, Daguang Xu, Jiashu Xu, Mengyao Xu, Ruqing Xu, Xingqian Xu, Yao Xu, Dinghao Yang, Dong Yang, Hans Yang, Xiaodong Yang, Xuning Yang, Yichu Yang, Yurong You, Zhiding Yu, Hao Yuan, Simon Yuen, Xiaohui Zeng, Pengcuo Zeren, Cindy Zha, Haotian Zhang, Jenny Zhang, Jing Zhang, Liangkai Zhang, Paris Zhang, Shun Zhang, Xuanmeng Zhang, Zhizheng Zhang, Ann Zhao, Yilin Zhao, Yuliya Zhautouskaya, Charles Zhou, Fengzhe Zhou, Shilin Zhu, Yuke Zhu, Dima Zhylko, and Artur Zolkowski. Cosmos 3: Omnimodal world models for physical ai, 2026. URL https://arxiv.org/abs/2606.02800.

[24] Runjia Qian, Zile Wang, Jihai Zhang, Kai Zou, Wei Yu, Jiaxing Li, Zexiang Liu, Yaokun Li, Fei Kang, Kaichen Huang, et al. Matrix-game 3.5: Enhancing real-time streaming interactive world models with patch memory. *arXiv preprint arXiv:2608.29910*, 2026.

[25] Rajit Rajpal, Shahbuland Matiana, Liew Wei Pyn, Anmol Agarwal, Ryan Craig, Andrew Lapp, Mithun Hunsur, Sami BuGhanem, Scottie Fox, Aaron Sanders, Carson Poole, Irene Park, Dave Rossi, Spencer Frazier, and Louis Castricato. Waypoint-1.5: A real-time video world model for consumer hardware, 2026. URL https://arxiv.org/abs/2609.37107.

[26] Georgy Savva, Oscar Michel, Daohan Lu, Suppakit Waiwitlikhit, Timothy Meehan, Dhairya Mishra, Srivats Poddar, Jack Lu, and Saining Xie. Solaris: Building a multiplayer video world model in minecraft, 2026. URL https://arxiv.org/abs/2602.22208.

[27] Wenqiang Sun, Haiyu Zhang, Haoyuan Wang, Junta Wu, Zehan Wang, Zhenwei Wang, Yunhong Wang, Jun Zhang, Tengfei Wang, and Chunchao Guo. Worldplay: Towards long-term geometric consistency for real-time interactive world modeling. *arXiv preprint arXiv:2512.14614*, 2025.

[28] AlayaWorld Team, Kaipeng Zhang, Chuanhao Li, Yifan Zhan, Yongtao Ge, Yuanyang Yin, Jiaming Tan, Kang He, Liaoyuan Fan, Ruicong Liu, et al. Alayaworld: Long-horizon and playable video world generation. *arXiv preprint arXiv:2607.06291*, 2026.

[29] DreamX Team, Yancheng Bai, Rui Chen, Xiangxiang Chu, Rujing Dang, Hao Dou, Bingjie Gao, Qiwen Gu, Siyu Hong, Jiachen Lei, Geng Li, Jifan Li, Ruimin Lin, Qingfeng Shi, Bingze Song, Lei Sun, Jing Tang, Ruitian Tian, Jun Wang, Jiahong Wu, Pengfei Zhang, Shen Zhang, and Jiashu Zhu. Dreamx-world 1.0: A general-purpose interactive world model, 2026. URL https://arxiv.org/abs/2606.16993.

[30] Kairos Team, Fei Wang, Shan You, Qiming Zhang, Tao Huang, Zuoyi Fu, Zhisheng Zheng, Yunlong Xi, Feng Lv, Xiaoming Wu, Zeyu Liu, Cong Wan, Pu Li, Ruiqing Yang, Xiaoou Li, Wei Wang, Kangkang Zhu, Yuwei Zhang, Shi Fu, Zheng Zhang, Xiaoning Wu, Xuzeng Fan, Dacheng Tao, and Xiaogang Wang. Kairos: A native world model stack for physical ai, 2026. URL https://arxiv.org/abs/2606.16533.

[31] Robbyant Team, Zelin Gao, Qiuyu Wang, Yanhong Zeng, Jiapeng Zhu, Ka Leong Cheng, Yixuan Li, Hanlin Wang, Yinghao Xu, Shuailei Ma, Yihang Chen, Jie Liu, Yansong Cheng, Yao Yao, Jiayi Zhu, Yihao Meng, Kecheng Zheng, Qingyan Bai, Jingye Chen, Zehong Shen, Yue Yu, Xing Zhu, Yujun Shen, and Hao Ouyang. Advancing open-source world models, 2026. URL https://arxiv.org/abs/2601.20540.

[32] Dani Valevski, Yaniv Leviathan, Moab Arar, and Shlomi Fruchter. Diffusion models are real-time game engines. In *International Conference on Learning Representations*, volume 2025, pages 73754–73776, 2025.

[33] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, et al. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.

[34] Zile Wang, Zexiang Liu, Jiaxing Li, Kaichen Huang, Baixin Xu, Fei Kang, Mengyin An, Peiyu Wang, Biao Jiang, Yichen Wei, et al. Matrix-game 3.0: Real-time and streaming interactive world model with long-horizon memory. *arXiv preprint arXiv:2604.08995*, 2026.

[35] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, volume 2024, pages 21875–21895, 2024.

[36] Jiacong Xu, Hanwen Jiang, Zhixin Shu, Kalyan Sunkavalli, Vishal M Patel, and Yiqun Mei. Wonder: Video world model done better. *arXiv preprint arXiv:2607.26037*, 2026.

[37] Shuai Yang, Wei Huang, Ruihang Chu, Yicheng Xiao, Yuyang Zhao, Xianbang Wang, Muyang Li, Enze Xie, Yingcong Chen, Yao Lu, et al. Longlive: Real-time interactive long video generation. *arXiv preprint arXiv:2509.22622*, 2025.

[38] Tianwei Yin, Michaël Gharbi, Taesung Park, Richard Zhang, Eli Shechtman, Fredo Durand, and William T Freeman. Improved distribution matching distillation for fast image synthesis. *Advances in neural information processing systems*, 37:47455–47487, 2024.

[39] Tianwei Yin, Qiang Zhang, Richard Zhang, William T Freeman, Fredo Durand, Eli Shechtman, and Xun Huang. From slow bidirectional to fast autoregressive video diffusion models. In *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 22963–22974. IEEE, 2025.

[40] Jihai Zhang, Runjia Qian, Kaiqin Kong, Biao Jiang, Puyi Wang, Siru Chen, Tianle Li, Guanjie Chen, Junyi Wu, Chubin Chen, Yuanhang Li, Kaichen Huang, Haobo Zhang, Wei Yu, Guibin Chen, Yu Cheng, and Yahui Zhou. Matrix-game 4.0: Engine-grounded neural rendering for real-time interactive worlds. Technical report, 2026. URL https://matrix-game-v4-0.github.io/page/.

[41] Zhanchao Zhou, Tianyi Wu, Zhiyun Jiang, Fares Obeid, and Zhenzhong Lan. Value residual learning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 28341–28356, 2025.

[42] Haoyi Zhu, Haozhe Liu, Yuyang Zhao, Tian Ye, Junsong Chen, Jincheng Yu, Tong He, Song Han, and Enze Xie. Sana-wm: Efficient minute-scale world modeling with hybrid linear diffusion transformer, 2026. URL https://arxiv.org/abs/2605.15178.

[43] Jie Zhu, Mingyu Ding, Boqiang Duan, Leye Wang, and Jingdong Wang. Unveiling the secret of adaln-zero in diffusion transformer.

A Autoencoder

Waypoint-2 uses a video autoencoder that maps four consecutive frames, in patches of 48×40 pixels, to one 128-channel latent token. A 720×1280 frame therefore becomes a 15×32 grid of 480 tokens. This is a $180\times$ compression in values, compared with $96\times$ for the TAEHV autoencoder [6] used by Waypoint-1.5 [25], and it lets Waypoint-2 render $1.76\times$ as many pixels as Waypoint-1.5 with fewer spatial tokens per frame (480 versus 512). The decoder of the autoencoder runs in a streaming fashion: it conditions on the previously decoded frame and uses local neighborhood attention.

B Training Hyperparameters

All stages use NorMuon (learning rate 10^{-3}, momentum 0.95, weight decay 0.01) for 2D weight matrices and AdamW (learning rate 10^{-4}, $\beta = (0.9, 0.95)$, weight decay 0.01) for the remaining parameters, with noise levels drawn as in Section 3.

Table 10: Training hyperparameters by stage. Clip and window lengths are in latent frames. The global batch is the number of sequences per optimizer step. In Stage I a sequence is a pack of up to 64 images. Steps are optimizer steps. Stage III starts from the Stage II (short) model. The weights are tracked with an exponential moving average of decay 0.9999 per step in Stage I, and 0.99 every 100 steps afterwards (every 40 steps in Stage IV).

	I	**II (short)**	**II (long)**	**III (short)**	**III (long)**	**IV**
Attention across frames	–	bidirectional	bidirectional	causal	causal	causal
Window	–	8	64	8	64	64, plus 4 pinned
Clip length	1	8	64	8	64	144 to 256
Global batch	64	64	64	16	32	32
Decoder draws K	1	1	1	4	4	1
Self-Flow token ratio	0.2	0.1	0.1	off	off	off
Clean seed frames	–	up to 2	up to 2	–	–	–
Text dropout	0.1	0.1	0.1	0.1	0.1	0.1
Control dropout	–	0.1	0.5	0.1	0.1	0.1
Steps	350k	425k	50k	25k	50k	50k

C Additional Samples

Figure 10 shows the first frames that the distilled model generates from a prompt alone, with no seed image. Because the first frame is generated from the BOS context, the final video model still generates images from text.

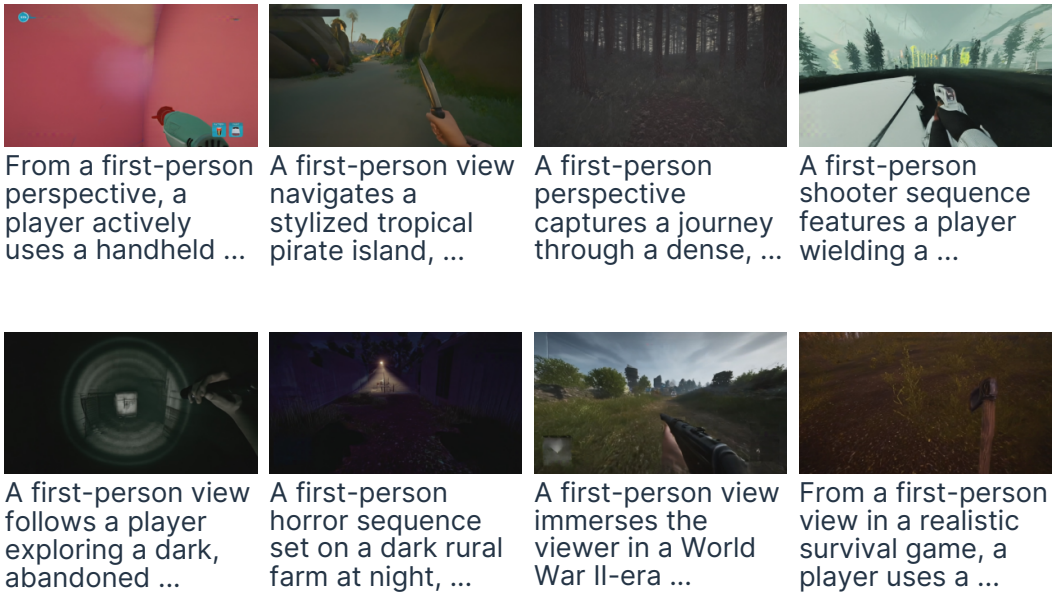


Figure 10: **Text-to-image samples from the distilled model.** First frames generated from a prompt, with no seed image, in four denoising steps at 720p.